

论文：[《Qwen3-VL Technical Report》](#)

代码仓库：[QwenLM/Qwen3-VL](#) | 完整PR代码：[代码链接](#) | 模型仓库：[Qwen/qwen3-vl](#)

25年9月我们在[Qwen3 VL Preview](#)中超前解析了 Qwen3-VL 的模型架构，当时依靠模型代码以及 [官方Blog](#) 无法对模型训练细节进行解析（并且当时没有开源全系列模型）。现在随着Qwen3-VL技术报告发布/模型完整开源，我们**重新来解析Qwen3-VL系列模型的关键技术！**

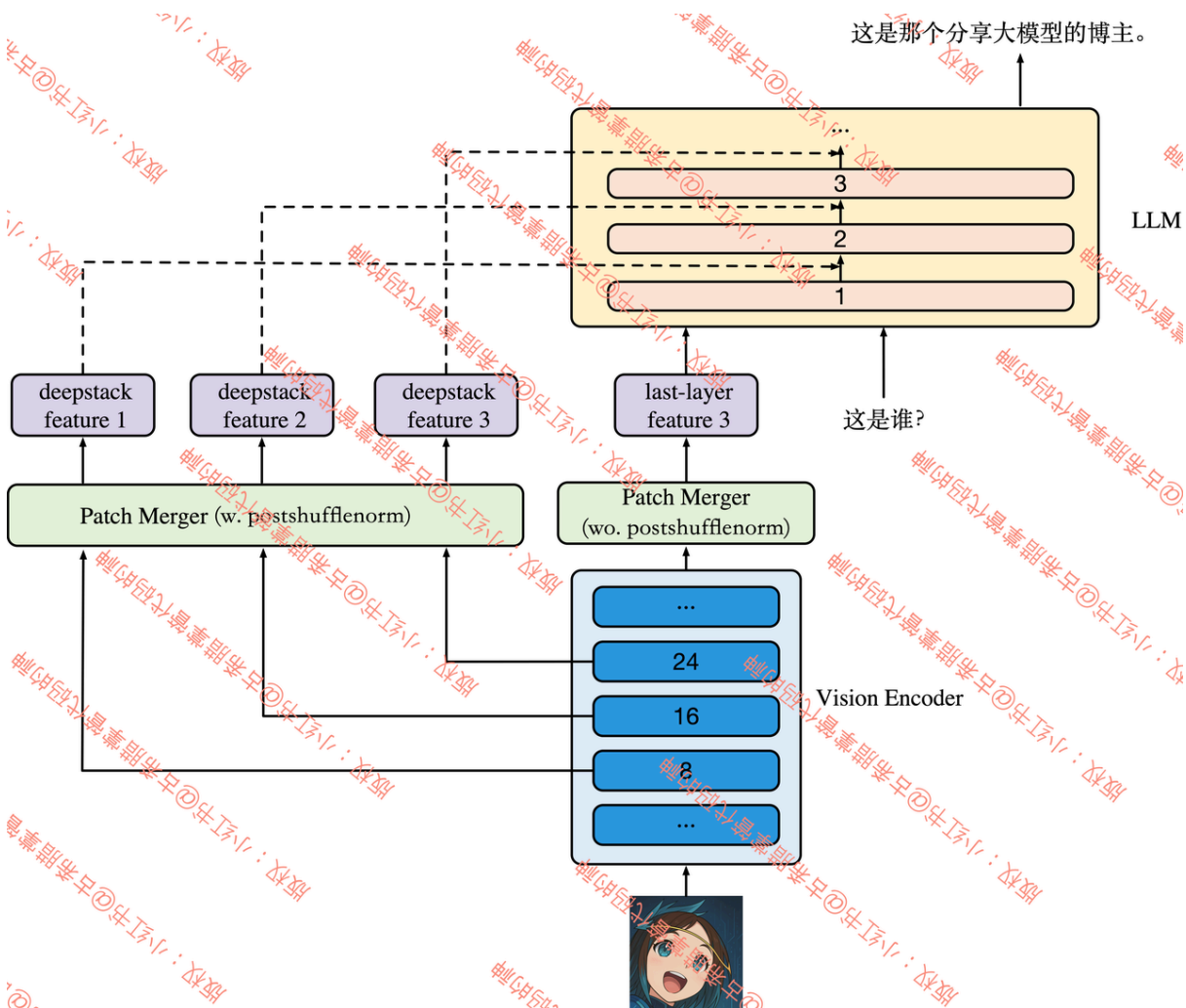
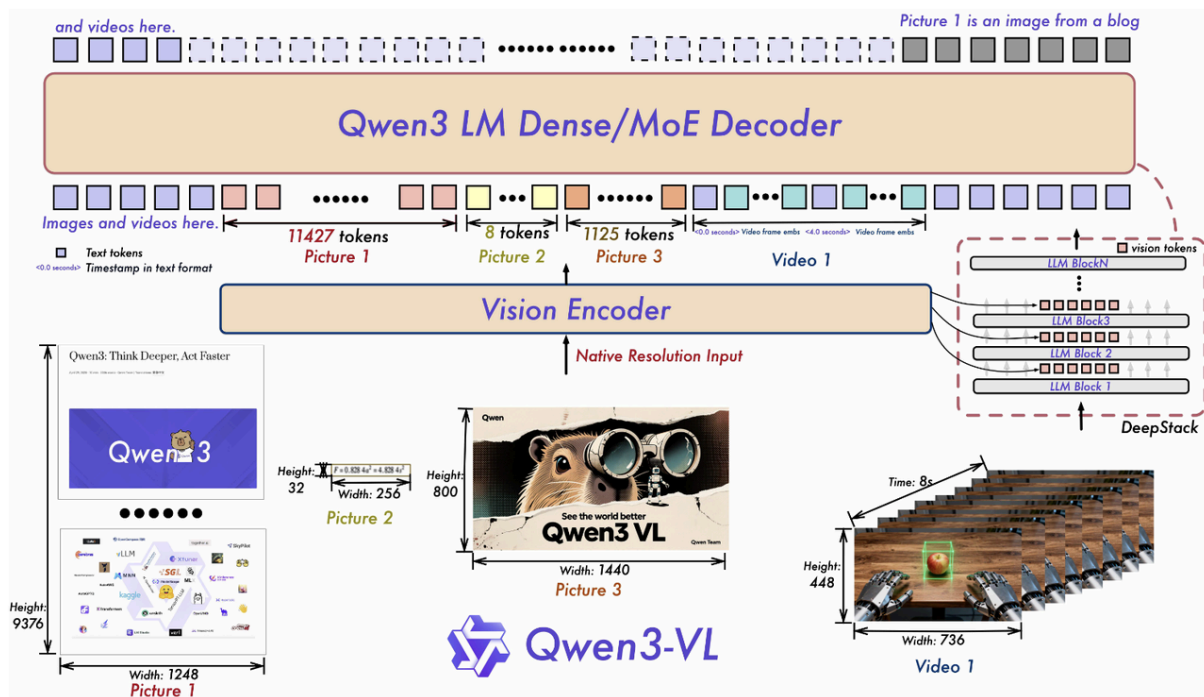
Qwen3-VL是Qwen系列目前最先进的视觉-语言多模态模型（截止2025年11月），核心特性与创新：

1. **架构主要改进（对比[Qwen2.5-VL](#)）**：使用的DeepStack的Projector整合多层次ViT特征提升视觉-语言对齐效果，LLM支持MoE和Dense不同架构。替换[MRoPE](#)为interleaved-MRoPE，强化图像/视频的时空特征建模的稳定性。
2. **长上下文支持**：原生支持256K tokens的长上下文，可无缝处理交错式文本、图像及视频输入。
3. **增强的后训练**：在多个细分领域进行RL，采用Qwen团队的SAPO，参考“think with images”多模态agent训练。

1. 模型架构

模型还是标准的“三段式结构”，即 **ViT+Projector+LLM**：

- **Vision Encoder**：[SigLIP2](#)架构，初始化于SigLIP2-SO-400M、SigLIP2-Large (300M)，进行动态分辨率CPT。沿用 Qwen2.5 bias 配置、[2D-RoPE](#)，但是替换了激活函数，PatchEmbed启用bias。
- **Projector**：Adaptor本次更新中使用了两个部分特征，一部分是沿用的Qwen2.5的 PatchMerger（对比解析参考[MLLM 之 MLP Projector 发展史](#)）另外一部分使用了 `deepstack_visual_indexes`，**用于与 LLM跨层进行特征融合。**
- **LLM**：[Qwen3-VL-235B-A22B](#) /Qwen3-VL-30B-A3B 使用的是 [Qwen 3](#)的MoE版本，32B/8B/4B/2B 使用的是 Qwen3 的Dense版本，架构上使用了QKNorm（解析参考[QK Norm 的前世今生](#)），与[Qwen2.5-VL](#)区别在于：Qwen3VL 改进了 [MRoPE](#)为 Interleaved MRoPE。**并且在前K层使用了DeepStack特征融合。**



1.1 Vision Encoder

Vision Encoder 使用的 [SigLIP2](#) 架构，ViT 权重来自于 SigLIP2-SO-400M 和 SigLIP2-Large (300M)，后者用于 **2B/4B** 的轻量级模型。Qwen3-VL 对 ViT 增加 [2D-RoPE](#) 后进行了动态分辨率的继续预训练，方法参考 [CoMP](#)。对比 [Qwen2.5-VL](#) 在 ViT 上的区别：

- PatchEmbed

Qwen3-VL: Qwen3VLVisionPatchEmbed 使用 Conv3d，**bias=True**，默认 patch_size=16。

Qwen2.5-VL: Qwen2_5_VisionPatchEmbed 也用 Conv3d，**bias=False**，默认 patch_size=14。

- 位置编码：

两者都使用 [2D-RoPE](#)，即支持动态分辨率（参考 [NaViT](#)）。

- MLP：

之前在 [Qwen2.5-VL](#) 中讲过，[Qwen2.5-VL](#) 的 ViT-MLP 做出了更像语言模型架构的策略：

- 使用 SwiGLU 的 FFN 结构，`hidden_act="silu"`（参考 [1.1.5 前馈网络 FFN 和激活函数 Activation](#)）
- 标准化使用 RMSNorm。
- 其中的线性层增加 bias，注意这里在 llama 这样的语言模型中是默认无偏置。

Qwen3VL 沿用了这一配置，除了激活函数：

```
class Qwen3VLVisionMLP(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.hidden_size = config.hidden_size
        self.intermediate_size = config.intermediate_size
        self.linear_fc1 = nn.Linear(self.hidden_size, self.intermediate_size, bias=True)
        self.linear_fc2 = nn.Linear(self.intermediate_size, self.hidden_size, bias=True)
        self.act_fn = ACT2FN[config.hidden_act]

    def forward(self, hidden_state):
        return self.linear_fc2(self.act_fn(self.linear_fc1(hidden_state)))
```

但是细节变化在于激活函数由 `silu` 替换为了 `gelu_pytorch_tanh`，参考代码：

```

class Qwen3VLVisionConfig(PretrainedConfig):
    model_type = "qwen3_vl"
    base_config_key = "vision_config"

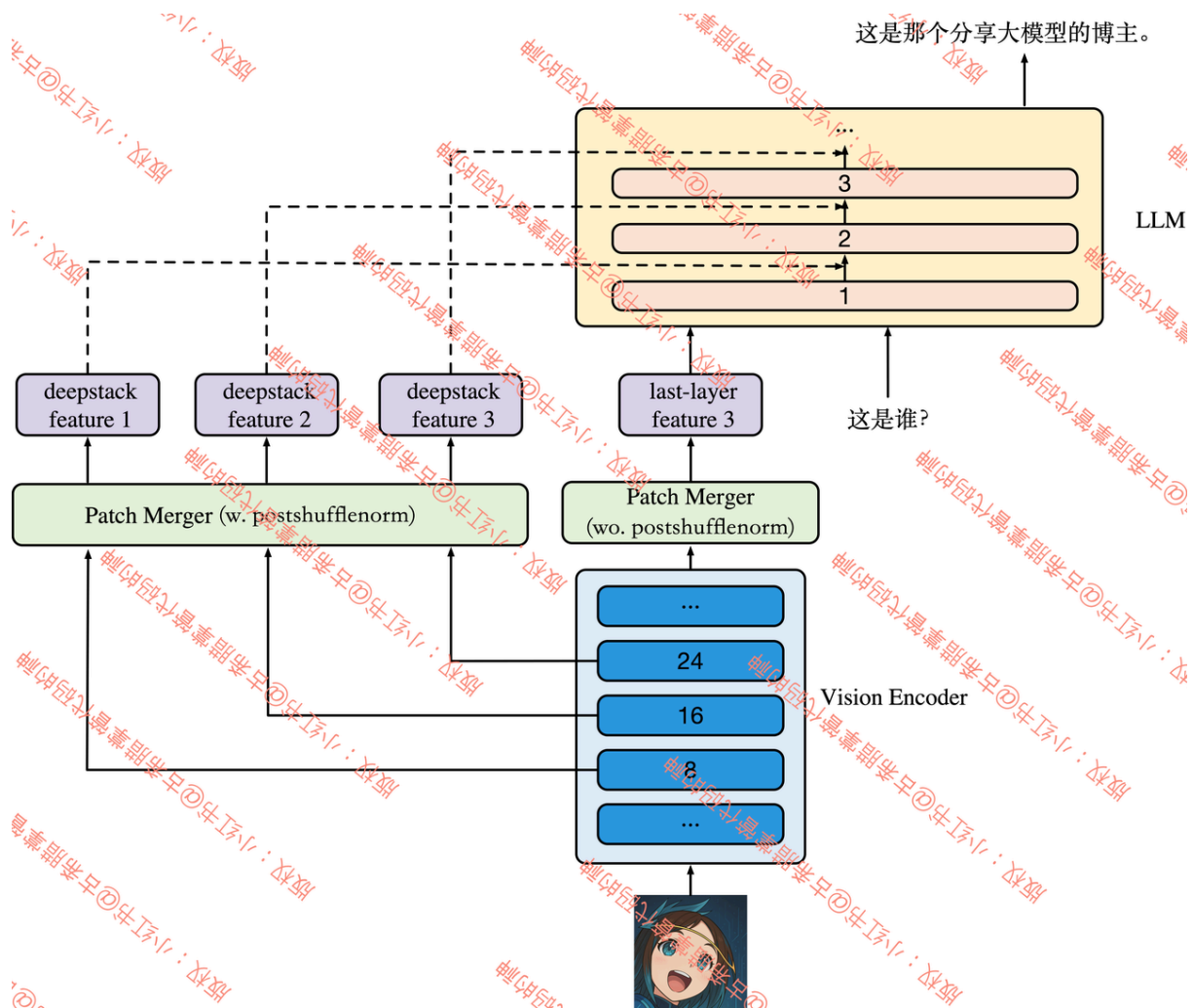
    def __init__(
        self,
        depth=27,
        hidden_size=1152,
        hidden_act="gelu_pytorch_tanh",
        intermediate_size=4304,
        num_heads=16,
        in_channels=3,
        patch_size=16,
        spatial_merge_size=2,
        temporal_patch_size=2,
        out_hidden_size=3584,
        num_position_embeddings=2304,
        deepstack_visual_indexes=[8, 16, 24],
        initializer_range=0.02,
        **kwargs,
    ):
        super().__init__(**kwargs)
        ...

```

1.1 Adaptor

Adaptor在沿用了 [Qwen2.5-VL](#) 的基础上，增加了从ViT中间层到LLM前k层到特征融合通路。总的来讲，Qwen3-VL的Adaptor包括两个部分：

- 一部分是沿用的Qwen2.5的 PatchMerger（对比解析参考 [MLLM 之 MLP Projector 发展史](#)）
- 另外一部分使用了 `deepstack_visual_indexes`，用于与LLM特征进行融合。如右图所示，通过特征通路进行视觉和语言的深度融合。



实现方式参考以下代码蓝色和红色部分：

```
class Qwen3VLVisionModel(Qwen3VLPreTrainedModel):
    config: Qwen3VLVisionConfig
    _no_split_modules = ["Qwen3VLVisionBlock"]

    def __init__(self, config, *inputs, **kwargs) -> None:
        super().__init__(config, *inputs, **kwargs)
        self.spatial_merge_size = config.spatial_merge_size
        self.patch_size = config.patch_size
        self.spatial_merge_unit = self.spatial_merge_size * self.spatial_merge_size

        self.patch_embed = Qwen3VLVisionPatchEmbed(
            config=config,
        )

        self.pos_embed = nn.Embedding(config.num_position_embeddings,
```

```

config.hidden_size)
    self.num_grid_per_side = int(config.num_position_embeddings**
0.5)

    head_dim = config.hidden_size // config.num_heads
    self.rotary_pos_emb = Qwen3VLVisionRotaryEmbedding(head_dim
// 2)

    self.blocks = nn.ModuleList([Qwen3VLVisionBlock(config) for _
in range(config.depth)])
    self.merger = Qwen3VLVisionPatchMerger(
        config=config,
        use_postshuffle_norm=False,
    )

    self.deepstack_visual_indexes = config.deepstack_visual_index
es
    # deepstack visual indexes=[8, 16, 24]
    self.deepstack_merger_list = nn.ModuleList(
        [
            Qwen3VLVisionPatchMerger(
                config=config,
                use_postshuffle_norm=True,
            )
            for _ in range(len(config.deepstack_visual_indexes))
        ]
    )

    self.gradient_checkpointing = False

```

可以看到，**deepstack_merger_list**分别针对ViT的第[8, 16, 24]层设置了**Qwen3VLVisionPatchMerger**。

但是这部分的关键参数设置在**use_postshuffle_norm**上区别于PatchMerger，两者的关键区别在于先做shuffle（这里的shuffle本质上是hidden_dim上进行2*2的堆叠，参考[Qwen2.5-VL](#)）还是先做LayerNorm：

```

class Qwen3VLVisionPatchMerger(nn.Module):
    def __init__(self, config: Qwen3VLVisionConfig, use_postshuffle_n
orm=False) -> None:
        super().__init__()

```

```

        self.hidden_size = config.hidden_size * (config.spatial_merge_
_size**2)
        self.use_postshuffle_norm = use_postshuffle_norm
        self.norm = nn.LayerNorm(self.hidden_size if use_postshuffle_
norm else config.hidden_size, eps=1e-6)
        self.linear_fc1 = nn.Linear(self.hidden_size, self.hidden_siz
e)
        self.act_fn = nn.GELU()
        self.linear_fc2 = nn.Linear(self.hidden_size, config.out_hidd
en_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.norm(x.view(-1, self.hidden_size) if self.use_postsh
uffle_norm else x).view(-1, self.hidden_size)
        x = self.linear_fc2(self.act_fn(self.linear_fc1(x)))
        return x

```

然后整体上看，Qwen3VLVisionModel（即ViT部分），提供给LLM的是两部分特征：

- hidden_states：最后一层通过PatchMerger的特征。
- **deepstack_feature_lists：第[8, 16, 24]层中间特征。**

代码参考，具体这两部分特征怎么用，我们看LLM部分代码：

```

class Qwen3VLVisionModel(Qwen3VLPreTrainedModel):
    def forward(self, hidden_states: torch.Tensor, grid_thw: torch.Te
nsor, **kwargs) -> torch.Tensor:
        ...
        deepstack_feature_lists = []
        for layer_num, blk in enumerate(self.blocks):
            hidden_states = blk(
                hidden_states,
                cu_seqlens=cu_seqlens,
                position_embeddings=position_embeddings,
                **kwargs,
            )
            if layer_num in self.deepstack_visual_indexes:
                # deepstack_visual_indexes=[8, 16, 24]
                deepstack_feature = self.deepstack_merger_list[self.d
eepstack_visual_indexes.index(layer_num)](
                    hidden_states

```

```

        )
        deepstack_feature_lists.append(deepstack_feature)

    hidden_states = self.merger(hidden_states)

    return hidden_states, deepstack_feature_lists

```

至于为什么要用中间特征，参考：[多模态LLM中adapter部分的设计，ViT的特征取哪层哪个位置？（字节-2面）](#)

1.2 LLM

分为[Qwen 3](#)的MoE版本/非MoE版本，具体开源的模型版本和LLM架构见表：

规格（架构）	模型名字
235B (MoE)	Qwen3-VL-235B-A22B-Instruct / Thinking / FP8 / GGUF
30B (MoE)	Qwen3-VL-30B-A3B-Instruct / Thinking / FP8 / GGUF
32B (Dense)	Qwen3-VL-32B-Instruct / Thinking / FP8 / GGUF
8B (Dense)	Qwen3-VL-8B-Instruct / Thinking / FP8 / GGUF
4B (Dense)	Qwen3-VL-4B-Instruct / Thinking / FP8 / GGUF
2B (Dense)	Qwen3-VL-2B-Instruct / Thinking / FP8 / GGUF

架构上使用了QKNorm（解析参考[QK Norm 的前世今生](#)），对于非MoE版本进行分析，区别在于：

- Qwen3VL 使用 3 维多模态位置编码 [MRoPE](#)，Qwen3 则使用标准 RoPE。
- Qwen3 支持 sliding window 层，Qwen3VL 未引入。
- **在前K层使用了DeepStack特征融合。**（详细参考 [Qwen3-VL的 DeepStack 技术是什么？](#)）

我们具体来看，ViT输出的特征如何与LLM特征进行融合的，首先ViT输出的特征是两部分：

- `hidden_states`：最后一层通过PatchMerger的特征。
- **`deepstack_feature_lists`：第[8, 16, 24]层中间特征。**

首先对于`hidden_states`，直接与文本特征拼接输入LLM：

```

class Qwen3VLModel(Qwen3VLPreTrainedModel):
    base_model_prefix = ""
    _checkpoint_conversion_mapping = {}
    # Reference: fix gemma3 grad acc #37208
    accepts_loss_kwargs = False

```



```

    config: Qwen3VLConfig
    _no_split_modules = ["Qwen3VLTextDecoderLayer", "Qwen3VLVisionBlock"]

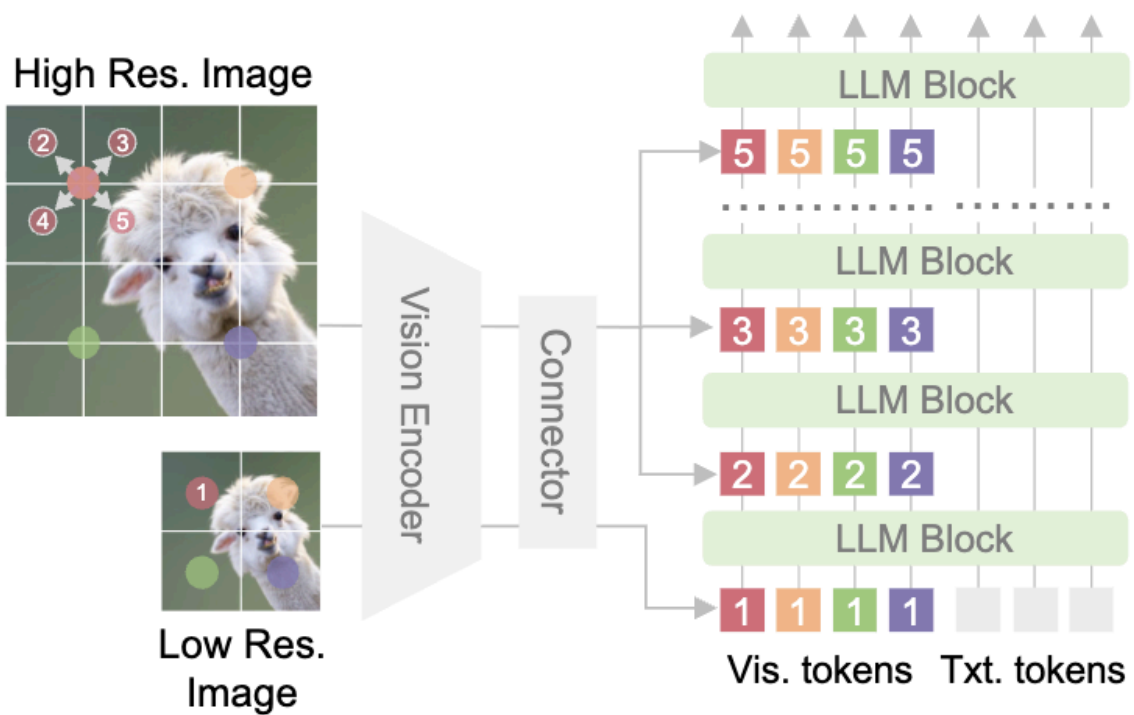
    def __init__(self, config):
        super().__init__(config)
        self.visual = Qwen3VLVisionModel._from_config(config.vision_config)
        self.language_model = Qwen3VLTextModel._from_config(config.text_config)
        self.rope_deltas = None # cache rope_deltas here

        # Initialize weights and apply final processing
        self.post_init()
    def forward(...):
        ...
        if pixel_values is not None:
            image_embeds, deepstack_image_embeds = self.get_image_features(pixel_values, image_grid_thw)
            image_embeds = torch.cat(image_embeds, dim=0).to(inputs_embeds.device, inputs_embeds.dtype)
            image_mask, _ = self.get_placeholder_mask(
                input_ids, inputs_embeds=inputs_embeds, image_features=image_embeds
            )
            inputs_embeds = inputs_embeds.masked_scatter(image_mask, image_embeds)
        ...

```

然后对于`deepstack_feature_lists`，则是分别与前K层（这里是前三层，`K=len(deepstack_feature_lists)`）进行融合。

我们首先来看这部分融合的代码逻辑：



```
class Qwen3VLTextModel(Qwen3VLPreTrainedModel):
    config: Qwen3VLTextConfig
    _no_split_modules = ["Qwen3VLTextDecoderLayer"]

    def __init__(self, config: Qwen3VLTextConfig):
        ...
        self.layers = nn.ModuleList(
            [Qwen3VLTextDecoderLayer(config, layer_idx) for layer_idx
in range(config.num_hidden_layers)]
        )
        ...

    def forward(
        ...
        visual_pos_masks: Optional[torch.Tensor] = None,
        deepstack_visual_embeds: Optional[list[torch.Tensor]] = None,
        ...
    ) -> Union[tuple, BaseModelOutputWithPast]:
        if (input_ids is None) ^ (inputs_embeds is not None):
            raise ValueError("You must specify exactly one of input_i
ds or inputs_embeds")

        ...
        # decoder layers
```

```

        for layer_idx, decoder_layer in enumerate(self.layers):
            layer_outputs = decoder_layer(
                hidden_states,
                attention_mask=attention_mask,
                position_ids=text_position_ids,
                past_key_values=past_key_values,
                cache_position=cache_position,
                position_embeddings=position_embeddings,
                **kwargs,
            )
            hidden_states = layer_outputs

        # add visual features to the hidden states of first several layers
        if deepstack_visual_embeds is not None and layer_idx in range(len(deepstack_visual_embeds)):
            hidden_states = self._deepstack_process(
                hidden_states,
                visual_pos_masks,
                deepstack_visual_embeds[layer_idx],
            )

        ...

```

然后融合的方式参考`_deepstack_process`函数，具体处理逻辑为：

- 把 `hidden_states` 在 `visual_pos_masks` 为 `True` 的位置切出来，加上对应层的视觉特征向量 `visual_embeds`（就是 `deepstack_visual_embeds[layer_idx]`，它已经对齐到“**只有视觉位置**”的子序列顺序）。
- 再写回到 `hidden_states` 的这些视觉位置。

这等价于：在若干前层（`deepstack` 的层数）里，用视觉中间层特征对语言隐藏态进行残差式强制注入，**仅作用于视觉 token 的那些位置，文本 token 位置不变**：

```

def _deepstack_process(
    self, hidden_states: torch.Tensor, visual_pos_masks: torch.Tensor, visual_embeds: torch.Tensor
):
    visual_pos_masks = visual_pos_masks.to(hidden_states.device)
    visual_embeds = visual_embeds.to(hidden_states.device, hidden_states.dtype)

```

```
local_this = hidden_states[visual_pos_masks, :].clone() + visual_embeds
hidden_states[visual_pos_masks, :] = local_this
return hidden_states
```

1.2.1 Interleaved MRoPE

鹅圈子有详细解析，麻烦查看：[链接](#)

2. 预训练

2.1 训练配置

Stage	Objective	Training	Token Budget	Sequence Length
S0	Vision-Language Alignment	Merger	67B	8,192
S1	Multimodal Pre-Training	All	~1T	8,192
S2	Long-Context Pre-Training	All	~1T	32,768
S3	Ultra-Long-Context Adaptation	All	100B	262,144

首先在S0之前需要对ViT进行CPT（继续预训练），主要目标是拓展其在动态分辨率下的能力，这部分不算在Stage里面。

1. Stage-0：视觉-语言对齐

- 目标：视觉与语言模态对齐
- 方法：仅训练MLP融合器，视觉编码器和LLM冻结
- 数据：67B tokens（高质量图文对、OCR、世界知识）

2. Stage-1：多模态预训练

- 目标：全参数端到端训练
- 方法：全参数训练，混合视觉Instruction数据与纯文本数据
- 数据：~1T tokens（含交错图文、视觉定位、VQA、STEM数据及少量视频）

3. Stage-2：长上下文预训练

- 目标：扩展上下文处理能力
- 调整：序列长度增至32768，增加纯文本比例和视频/任务数据
- 数据：~1T tokens（强化长视频与多步骤任务理解）

4. Stage-3：超长上下文适应

- 目标：突破上下文窗口极限
- 调整：序列长度扩展至262144
- 数据：100B tokens（聚焦长视频/长文档分析任务）

2.2 数据来源

数据合成方法论：

- 1) 强调模型标注（Qwen2.5VL系列）的数据合成、过滤与验证。
- 2) 合成数据与真实数据配比、多样性采样，平衡规模与质量。
- 3) 多阶段处理流程（生成→验证），确保数据可靠性。
- 4) 统一标注框架与标准化设计（如坐标、格式）提升泛化性。

• 图像-文本对数据 (Image-Caption Pairs)

使用 [Qwen2.5-VL](#)-32B模型对原始文本进行重新标注，生成更细粒度的描述（对象属性、空间布局等）。

基于视觉嵌入的聚类增强，对数据分布稀疏区域并进行定向增强。

• 交错文本-图像数据 (Interleaved Text-Image)

对（多模态）网页、书籍使用 [Qwen2.5-VL](#)-7B（轻量级模型）进行多模态解析，精确对齐文本与图表/照片。

低价值内容过滤：移除纯文本/未对齐片段，要求最低页面数、图像-文本比例。

最终需要切分、合并为最长不超过256K token的序列。

• 知识实体数据 (World Knowledge)

百科数据，覆盖10+语义类别（动物/植物/地标等）。

重要性采样策略：高显著性实体高频采样，低显著性实体低频保留。

使用LLM生成增强描述，包含视觉属性、空间关系和交互信息。

• OCR与文档解析

OCR：30M内部样本 + 30M合成多语言样本，均使用Qwen2.5-VL标注。

文档解析：3M PDF + 4M内部文档，使用布局模型预测阅读顺序，Qwen2.5-VL-72B进行区域识别（类似 [PaddleOCR-VL](#)）。

• 视觉定位与计数 (Grounding and Counting)

融合开源数据与合成数据构建框/点定位数据集，通过 [Qwen2.5-VL](#) 进行标注/低置信度过滤。

计数任务涵盖直接计数、先给出框/点再引导计数、[0,1000]先坐标标准化后再计数三种形式。

- **空间与3D理解 (Spatial Understanding and 3D Recognition)**

相比视觉定位与计数，3D理解需要推理空间关系、功能推断（如“可抓握”“可按压”）以及可行动作序列（如“如何移动障碍物拿到书”），目标是为具身智能（Embodied AI）和交互系统提供基础，支持对3D场景的深层语义理解。

空间理解数据合成：融合真实场景与合成布局数据，通过模板和LLM生成多样化自然语言查询，强调相对空间关系表达（非绝对坐标）。

3D定位数据合成：整合多源公共场景数据，构建单目图像+自然语言指代+3D边界框（9自由度）的问答格式数据集。过滤遮挡/噪声标签，统一至虚拟相机坐标系（基于Omni3D方法），确保空间一致性。合成细粒度描述文本，涵盖物体属性、布局、空间位置及交互关系，提升指代表达精准度。

- **多模态代码能力 (Code)**

复用Qwen3-Coder系列大规模代码语料。

开源整理+合成多模态Coding数据集：UI转HTML/CSS、SVG生成、流程图转代码等。

- **视频理解 (Video)**

在视频数据集的内容多样性上进行均衡采样。

FPS动态采样策略：根据序列长度约束调整fps和最大帧数。

利用字幕模型进行密集标注，合成时间连贯的 caption label。

- **STEM多模态推理 (Science, Technology, Engineering, and Mathematics)**

使用代码生成图示的方法生成几何图表数据：1M点标注样本 + 2M视觉问答对，并使用reward model进行检查。

多模态推理：60M K-12/本科习题 + 12M合成推理样本，通过**拒绝采样**仅保留具有挑战性的问题。

- **代理数据 (Agent)**

GUI交互数据涵盖跨平台界面感知与多步任务轨迹：在跨平台界面感知方面，通过元数据、解析工具和人工标注构建了诸如元素描述、密集字幕生成和密集定位等任务，从而实现对多样化用户界面的稳健理解。在多步任务方面，通过一个自我演化的轨迹生成框架组装多步骤任务轨迹，并辅以针对性的人工审核。

函数调用通过合成Pipeline实现无需真实执行的数据合成：首先通过图像指导具备能力的模型生成用户查询及其对应的功能定义。接着我们采样模型的功能调用并生成其推理过程，并合成功能响应。该过程会不断重复，直到用户的查询被判断为已解决，剔除其中格式错误的轨迹。

搜索能力集成在线工具，支持长尾实体知识获取：通过在线图像搜索和文本搜索工具收集多模态事实查询轨迹，鼓励模型对不熟悉的实体进行搜索。通过这种方式，模型学会了从网络上收集信息，从而生成更加准确的响应。

3. 后训练

后训练流程分为三个阶段，旨在提升模型的指令遵循、推理能力及人类偏好对齐：

Stage 1：监督微调（SFT）

- 目标：激活指令遵循能力与潜在推理技能。
- 实现方式：
 - 两阶段训练：
 1. 初始阶段：基于32k上下文长度进行训练。
 2. 扩展阶段：将上下文窗口扩展至256k，专注于长文档和长视频数据的处理。
 - 数据分类：
 - 标准格式：适用于非思考模型（Instruct模型）。
 - 思维链（CoT）格式：显式建模推理过程，适用于思考模型（Thinking模型）。

Stage 2：强到弱蒸馏（Strong-to-Weak Distillation）

- 核心方法：通过知识蒸馏，将强教师模型的能力迁移至学生模型。
- 关键设计：
 - 纯文本数据微调：仅使用纯文本数据对LLM主干进行优化。
 - 效果：显著提升文本任务与多模态任务的推理能力。

Stage 3：强化学习（RL）

- 目标：增强模型性能与人类偏好对齐，优化细粒度能力。
- 子阶段划分：
 - 推理RL：针对数学、OCR、基础推理等需逻辑推导的领域。

- 通用RL：覆盖多模态任务（如指令遵循、文本-图像对齐等）。

3.1 SFT

这部分方法是确定的，就是做Instruction Tuning。关键在于数据构建，分为两个部分：Instruct模型（非思考模型）数据构建、Thinking模型（思考模型）数据构建。

Instruct 模型SFT数据

在[Qwen2.5-VL](#)的8大核心领域、30个类别基础上进行拓展，新增空间推理（具身智能）、细粒度视觉理解、视频时空定位（目标追踪）及长上下文技术文档解析等类别的数据。

- **数据规模与组成**：SFT数据集包含约120万样本，其中：
 - **单模态数据**（1/3）：纯文本数据条目。
 - **多模态数据**（2/3）：图像-文本对、视频-文本对，支持复杂场景解析。
- **分阶段训练**：为适配256K token长上下文能力，采用两阶段训练：
 - **第一阶段**：32K token序列长度，训练1个epoch；
 - **第二阶段**：256K token全长度，混合长上下文数据与32K token样本（[采用课程学习策略](#)）。

采用 **两阶段过滤** 对查询（query）和响应（response）进行过滤，一方面需要保证查询足够清晰、可验证。另外一方面需要保证响应正确、无害。

- **查询过滤**：
 - 使用[Qwen2.5-VL过滤](#)不可验证或模糊指令。
 - 修订歧义查询，保留语义意图。
 - 剔除低信息密度网络查询，保留高复杂度、高相关性样本。
- **响应过滤**：
 - **规则过滤（Rule-Based Filtering）**：去除重复、不完整、格式错误或有害内容。
 - **模型过滤（Model-Based Filtering）**：基于Qwen2.5-VL的奖励模型多维度评估。
 - **答案质量**：正确性、完整性、清晰度、实用性。
 - **视觉任务**：重点验证视觉信息利用的准确性。
 - **隐蔽问题检测**：识别语言混杂、风格突变等规则难以覆盖的缺陷。

思考模型 Long-CoT 冷启动数据

为了对Thinking模型进行冷启动SFT，构建专门的长思维链（Long-CoT）冷启动数据集，其中多模态与纯文本数据1:1混合：

1. 数据构成

- 多模态数据（占比50%）：聚焦视觉问答（VQA）、OCR、2D/3D定位、视频分析等传统任务，强化STEM与智能体工作流相关的高阶推理能力。
- 纯文本数据（占比50%）：延续Qwen3的数据特征，覆盖数学、代码生成、逻辑推理等复杂学科领域。

2. 过滤低质、简单样本方法

- **难度校准**：我们选择性地保留基线模型通过率较低或生成较长、更详细回复的实例。这使得数据集中包含了对当前模型而言真正具有挑战性的问题。
- **多模态必要性筛选**：通过Qwen3-30B-nothink模型过滤可仅凭文本解决的视觉数学问题，保证视觉输入的必要性。
- **响应净化机制**：
 - 答案核对：优先剔除含错误答案的候选答案。
 - 过程核对：二次过滤存在重复表达、语言混杂、缺乏逻辑推演的猜测型回答。

3.2 Strong-to-Weak Distillation

Strong-to-Weak Distillation（强到弱蒸馏）流程是为了使用旗舰模型提升轻量级模型的性能：

- **Off-policy Distillation**：在第一阶段，直接使用Teacher模型输出SFT轻量级的Student模型，硬蒸馏。
- **On-policy Distillation**：在第二阶段，学生模型根据提供的提示生成响应，最小化 KL 散度来对齐学生模型和教师模型的预测结果（logits），软蒸馏。

3.3 RL

推理强化学习（Reasoning RL）

数据准备

- 来源：开源与私有数据，经严格预处理和人工标注确保质量，**注意收集的问题都是可以被规则和代码验证的。**

- 多模态数据筛选：通过预训练视觉语言模型采样16个响应，全错则丢弃查询；过滤通过率超90%的简单任务，混合多任务批次并预设任务样本比例。（类似DAPO的过滤策略）
- 实验筛选：通过初步RL实验剔除低潜力数据，最终保留约3万条多任务query。

奖励设计

- 统一框架，对每一个任务单独设计Reward，这部分细节保密了。
- 唯一提供的细节：通过提示控制输出格式，避免显式格式奖励；对语言切换（如非提示语言响应）施加惩罚。

RL 算法

- 采用SAPO算法（自适应策略梯度法，同样来自Qwen团队）。

通用强化学习（General RL）

目标：

- 1) 增强模型泛化能力与鲁棒性，纠正SFT阶段的错误先验知识（如反直觉计数、复杂时钟识别）。
- 2) 抑制有害行为：语言混合、重复输出、格式错误（通过专用数据集高频惩罚优化）。

奖励机制

- 指令遵循：评估内容、格式、长度等约束的匹配度（如JSON结构化输出）。
- 偏好对齐：优化开放性任务的主观质量（有用性、事实性、风格适配性）。

混合奖励系统

- 规则奖励：针对可验证任务（如格式遵循），用明确启发式规则减少奖励欺骗风险。
- 模型奖励：Qwen2.5-VL-72B或Qwen3作为Reward Model 评估开放任务，减少严格规则导致的误判（如非标准但有效响应）。

3.4 Thinking with Images

按照之前"Thinking with images"的思路，如果给VLM赋予Agent能力，能帮助其在Reasoning上进一步Scaling，因此Qwen3-VL提出了一种两阶段训练范式，赋予Qwen3-VL视觉Agent能力：

1. 第一阶段

- 合成约10k冷启动基础数据集，主要包含两轮视觉问答任务（如attribute detection）。
- 通过监督微调(SFT)训练Qwen2.5-VL-32B，模拟"思考→行动→反馈分析→回答"的Agent行为。
- 采用**multi-turn, tool-integrated**强化学习(RL)增强推理能力。

2. 第二阶段

- 通过第一阶段模型生成更大规模的120k多轮交互数据集。
- 结合合成与蒸馏数据，采用类似SFT+RL流程进行后训练。

两个阶段共享奖励设计（唯独数据不同）：

- **答案准确度奖励**：基于Qwen3-32B评估最终答案正确性。
- **多轮推理奖励**：通过Qwen2.5-VL-72B验证工具反馈解析与分步推理的连贯性。
- **工具调用奖励**：动态匹配工具使用次数与专家预估目标值（由Qwen2.5-VL-72B离线评估任务复杂度确定），早期的实验表明，**模型倾向于退化为仅调用一次工具来获取前两个奖励**，而不管任务需求如何。为了缓解这一问题，我们明确地将工具调用奖励纳入其中，以促进与任务复杂度相适应的工具探索。

4. 实验

通用视觉问答（VQA）

- Qwen3-VL系列在不同规模模型（2B至235B参数）中均展现竞争力。旗舰模型Qwen3-VL-235B-A22B-Thinking在MMStar基准测试中达到78.7分，达到SOTA。
- 中小型模型（如32B）在RealWorldQA等任务中超越Gemini 2.5 Flash、GPT-5 Mini模型，显示良好的扩展性。
- 可以参考 [Gemini-3-Pro](#) 官网的benchmark得分，感觉差距还是有点大的，我指的是不如 [Gemini-3-Pro](#)。

	Benchmark	Qwen3-VL 235B-A22B		Gemini 2.5 Pro		OpenAI GPT-5		Claude Opus 4.1	
		thinking	instruct	thinking	budget-128	high	minimal	thinking	non-thinking
STEM Puzzle	MMMU	80.6	78.7	81.7*	<u>80.9</u>	84.2*	74.4*	78.4	77.2
	MMMU-Pro	69.3	68.1	68.8*	<u>71.2</u>	78.4*	62.7*	64.8	60.7
	MathVista _{mini}	85.8	<u>84.9</u>	82.7*	<u>77.7</u>	81.3	50.9	75.5	74.5
	MathVision	74.6	<u>66.5</u>	73.3*	66.0	70.9	45.8	64.3	57.7
	MathVision _{WP}	63.8	<u>57.0</u>	63.2	56.9	62.8	40.1	54.0	46.4
	We-Math	74.8	<u>67.5</u>	80.6	<u>74.5</u>	73.8	51.8	65.2	60.2
	MathVerse _{mini}	85.0	<u>72.5</u>	82.9	65.9	84.1	43.0	70.6	68.1
	DynaMath	82.8	<u>79.4</u>	80.0	78.5	85.4	74.0	75.1	72.0
	Math-VR	66.8	<u>65.0</u>	64.7*	54.3	58.1	21.7	54.3	38.0
	ZeroBench	4	<u>2</u>	3	1	2	<u>2</u>	3	1
	VlmsAreBlind	79.5	<u>80.4</u>	86.1	78.5	80.5	53.4	77.8	72.2
	LogicVista	72.2	<u>65.8</u>	72.0	<u>68.7</u>	71.8	46.3	67.3	63.5
	VisuLogic	34.4	<u>29.9</u>	31.6	<u>26.9</u>	28.5	27.2	27.9	27.2
	VisualPuzzles	57.2	<u>54.7</u>	60.9	<u>56.9</u>	57.3	47.9	48.8	47.6
General VQA	MMBench-EN	88.8	<u>89.3</u>	90.1*	88.4	83.8	81.3	79.4	83.0
	MMBench-CN	88.6	<u>88.9</u>	89.7*	86.4	83.5	79.9	84.9	74.3
	RealWorldQA	81.3	<u>79.2</u>	78.0*	76.0	82.8	77.3	69.9	68.5
	MMStar	78.7	<u>78.4</u>	77.5*	<u>78.5</u>	76.4	65.2	72.1	71.0
	SimpleVQA	61.3	63.0	65.4	<u>66.9</u>	61.8	56.7	56.7	55.7
Alignment	HallusionBench	66.7	<u>63.2</u>	63.7*	60.9	65.7	53.7	60.4	55.1
	MM-MT-Bench	8.5	<u>8.5</u>	8.4*	7.6	7.6	7.5	7.8	7.9
	MIA-Bench	92.7	<u>91.3</u>	92.3	91.3	92.4	<u>92.6</u>	91.2	90.0
Document Understanding	DocVQA _{test}	96.5	<u>97.1</u>	92.6	94.0	91.5	89.6	92.5	89.2
	InfoVQA _{test}	89.5	<u>89.2</u>	84.2	82.9	79.0	69.9	69.4	60.9
	AI2D _{w. M.}	89.2	<u>89.7</u>	90.9	<u>90.0</u>	89.7	84.1	86.4	84.4
	ChartQA _{test}	90.3	<u>90.3</u>	83.3	62.6	59.7	59.1	86.2	83.9
	OCRBench	875	<u>920</u>	866	872	810	787	764	750
	OCRBench_v2 _{en}	66.8	<u>67.1</u>	54.3	55.2	53.0	48.2	48.4	47.2
	OCRBench_v2 _{zh}	63.5	<u>61.8</u>	48.5	53.1	43.2	37.7	43.7	38.0
	CC-OCR	81.5	<u>82.2</u>	77.2	76.8	68.3	66.1	69.1	66.0
	OmniDocBench _{en}	0.155	<u>0.143</u>	0.347	0.206	0.356	0.174	0.194	-
	OmniDocBench _{zh}	0.207	<u>0.207</u>	0.238	0.249	0.472	0.389	0.293	-
	CharXiv(DQ)	90.5	<u>89.4</u>	94.4	87.8	89.2	79.5	88.5	87.8
	CharXiv(RQ)	66.1	<u>62.1</u>	67.9	62.9	81.1*	57.8	63.6	60.2
	MMLongBench _{Doc}	56.2	<u>57.0</u>	55.6	51.2	51.5	42.4	54.5	48.1
2D/3D Grounding	RefCOCO-avg	92.1	<u>91.9</u>	74.6*	-	66.8	-	-	-
	CountBench	93.7	<u>93.0</u>	91.0*	91.0	91.7	87.8	93.1	91.9
	ODinW-13	43.2	<u>48.6</u>	33.7*	34.5	-	-	-	-
	ARKitScenes	53.7	<u>56.9</u>	-	-	-	-	-	-
	Hypersim	11.0	<u>13.0</u>	-	-	-	-	-	-
	SUNRGBD	34.9	<u>39.4</u>	29.7	-	-	-	-	-
Embodied/Spatial Understanding	ERQA	52.5	<u>51.3</u>	55.3	50.3	65.7*	42.0*	34.8	28.0
	VSI-Bench	60.0	<u>62.7</u>	-	-	-	-	-	-
	EmbSpatialBench	84.3	<u>83.1</u>	79.1	73.3	82.9	75.1	69.2	66.0
	RefSpatialBench	69.9	<u>65.5</u>	36.5	35.6	23.8	23.1	-	-
Multi-Image	RoboSpatialHome	73.9	<u>69.4</u>	47.5	49.2	53.5	43.6	-	-
	BLINK	67.1	<u>70.7</u>	70.6*	70.0	71.0	62.8	64.1	62.9
Video Understanding	MUIRBENCH	80.1	<u>73.0</u>	77.2	<u>74.0</u>	77.5	66.5	-	-
	MVBench	75.2	<u>76.5</u>	69.9	65.8	75.3	64.6	61.4	59.0
	Video-MME _{w/o sub.}	79.0	<u>79.2</u>	85.1	<u>80.6</u>	84.7	77.3	75.6	73.3
	MLVU _{M-Avg}	83.8	<u>84.3</u>	85.6	81.2	86.2	78.3	73.5	71.2
	LVBench	63.6	<u>67.7</u>	73.0	<u>69.0</u>	-	-	-	-
	Charades-STA _{mIoU}	63.5	<u>64.8</u>	-	-	-	-	-	-
	VideoMMMU	80.0	<u>74.7</u>	83.6*	<u>79.4</u>	84.6*	61.6*	76.2	70.1
Perception with Tool	MMVU	71.1	<u>68.1</u>	74.9	<u>72.2</u>	73.0	68.1	66.4	61.4
	V*	85.9	<u>93.7+</u>	83.8	72.7	72.8	56.7	-	-
	HRBench4K	84.3	<u>85.4+</u>	87.3	84.8	-	-	-	-
Multi-Modal Coding	HRBench8K	76.6	<u>82.4+</u>	85.4	80.1	-	-	-	-
	Design2Code	93.4	<u>92.0</u>	89.2	90.3	92.5	88.9	88.5	85.3
	ChartMimic	78.4	<u>80.5</u>	83.9	79.9	62.1	41.4	85.2	<u>82.9</u>
Multi-Modal Agent	UniSVG	65.8	<u>69.8</u>	70.0	67.9	71.7	<u>74.5</u>	73.0	<u>72.5</u>
	ScreenSpot Pro	61.8	<u>62.0</u>	-	-	-	-	-	-
	OSWorldG	68.3	<u>66.7</u>	45.2	-	-	-	-	-
	AndroidWorld	62.0	<u>63.7</u>	-	-	-	-	-	-
	OSWorld	38.1	<u>31.6</u>	-	-	-	-	-	<u>44.4</u>
	WindowsAA	32.1	<u>28.9</u>	-	-	-	-	-	-

	Benchmark	Qwen3-VL 30B-A3B		Qwen3-VL 32B		Gemini 2.5 Flash		GPT-5 mini	
		thinking	instruct	thinking	instruct	thinking	non-thinking	high	minimal
STEM Puzzle	MMMU	76.0	74.2	78.1	76.0	77.7	76.3	79.0	67.9
	MMMU-Pro	63.0	60.4	68.1	65.3	67.2	65.9	67.3	53.7
	MathVista _{mini}	81.9	80.1	85.9	83.8	79.4	75.3	79.1	59.6
	MathVision	65.7	60.2	70.2	63.4	64.3	60.7	71.9	46.6
	MathVision _{WP}	58.9	52.3	58.6	54.6	53.6	49.0	56.6	42.8
	We-Math	70.0	56.9	71.6	63.3	53.9	60.3	70.2	51.4
	MathVerse _{mini}	79.6	70.2	82.6	76.8	77.7	75.9	78.8	36.5
	DynaMath	80.1	73.4	82.0	76.7	75.9	69.7	81.4	71.3
	Math-VR	61.7	61.3	62.3	59.8	58.8	54.7	58.2	26.4
	ZeroBench	0	0	2	1	1	3	3	2
	VlmsAreBlind	72.5	67.5	85.1	87.0	77.5	75.9	75.8	62.0
	LogicVista	65.8	53.5	70.9	62.2	67.3	60.0	71.4	50.8
	VisuLogic	26.6	23.0	32.4	29.7	31.0	23.3	27.2	27.6
	VisualPuzzles	52.0	46.2	54.7	53.2	41.4	45.0	59.3	48.2
General VQA	MMBench-EN	87.0	86.1	89.5	87.6	87.1	86.6	86.6	78.5
	MMBench-CN	85.9	85.3	89.4	87.7	87.3	86.0	84.0	76.3
	RealWorldQA	77.4	73.7	78.4	79.0	76.0	75.7	79.0	73.3
	MMStar	75.5	72.1	79.4	77.7	76.5	75.8	74.1	61.3
	SimpleVQA	54.3	52.7	55.4	56.9	63.2	59.2	56.8	50.3
Alignment	HallusionBench	66.0	61.5	67.4	63.8	63.5	59.1	63.2	55.9
	MM-MT-Bench	7.9	8.0	8.3	8.4	8.1	8.0	7.7	7.4
	MIA-Bench	91.6	91.2	92.3	91.8	91.1	90.6	92.0	92.3
Document Understanding	DocVQA _{test}	95.5	95.0	96.1	96.9	92.8	93.0	90.5	90.6
	InfoVQA _{test}	85.6	81.8	89.2	87.0	82.5	81.7	77.6	72.8
	AI2D _{w. M.}	86.9	85.0	88.9	89.5	88.7	87.7	88.2	82.9
	ChartQA _{test}	89.4	86.8	89.0	88.5	60.6	69.0	57.5	57.8
	OCRBench	839	903	855	895	853	864	821	807
	OCRBench_v2 _{en}	62.6	63.2	68.4	67.4	52.2	50.6	52.6	45.7
	OCRBench_v2 _{zh}	60.4	57.8	62.1	59.2	43.8	43.9	45.1	41.0
	CC-OCR	77.8	80.7	79.6	80.3	75.4	74.8	70.8	61.6
	OmniDocBench _{en}	0.165	0.183	0.148	0.151	0.265	0.228	0.181	0.260
	OmniDocBench _{zh}	0.233	0.253	0.236	0.239	0.245	0.305	0.316	0.425
	CharXiv(DQ)	86.9	85.5	90.2	90.5	90.1	85.5	89.4	78.6
	CharXiv(RQ)	56.6	48.9	65.2	62.8	61.7	60.1	68.6	48.9
	MMLongBench _{Doc}	47.4	47.1	54.6	55.4	49.0	44.6	50.3	39.6
2D/3D Grounding	RefCOCO-avg	89.3	89.7	91.1	91.9	-	-	-	-
	CountBench	90.0	89.8	94.1	94.9	86.0	83.7	91.0	84.1
	ODinW-13	42.3	47.5	41.8	46.6	-	-	-	-
	ARKitScenes	55.6	56.1	46.1	55.6	-	-	-	-
	Hypersim	11.4	12.5	12.5	14.0	-	-	-	-
	SUNRGBD	34.6	38.1	33.9	37.0	-	-	-	-
Embodied/Spatial Understanding	ERQA	45.3	43.0	52.3	48.8	-	-	54.0	45.8
	VSI-Bench	56.1	63.2	61.2	61.5	-	-	31.5	30.5
	EmbSpatialBench	80.6	76.4	82.7	81.5	-	-	80.7	72.1
	RefSpatialBench	54.2	53.1	67.2	61.4	-	-	9.0	4.0
	RoboSpatialHome	65.5	62.9	74.2	64.6	-	-	54.3	44.6
Multi-Image	BLINK	65.4	67.7	68.5	67.3	68.1	66.8	-	56.7
	MUIRBENCH	77.6	62.9	80.3	72.8	72.7	67.5	-	57.5
Video Understanding	MVBench	72.0	72.3	73.2	72.8	-	-	-	-
	Video-MME _{w/o sub.}	73.3	74.5	77.3	76.6	79.6	75.6	78.9	71.0
	MLVU _{M-Avg}	78.9	81.3	82.3	82.1	82.1	77.8	83.3	71.7
	LVBench	59.2	62.5	62.6	63.8	64.5	62.2	-	-
	Charades-STA _{mIoU}	62.7	63.5	62.8	61.2	-	-	-	-
	VideoMMMU	75.0	68.7	79.0	71.9	73.9	65.2	82.5*	56.7
	MMVU	66.1	59.8	67.9	66.8	69.8	68.2	69.8	64.8
Perception with Tool	V*	81.2	89.5 ⁺	84.8	91.1 ⁺	-	-	78.6	63.9
	HRBench4K	77.8	82.5 ⁺	82.1	84.6 ⁺	-	-	78.6	66.3
	HRBench8K	71.3	79.3 ⁺	74.8	81.6 ⁺	-	-	74.4	60.9
Multi-Modal Agent	ScreenSpot Pro	57.3	60.5	57.1	57.9	-	-	-	-
	OSWorldG	59.6	61.0	64.0	65.1	-	-	-	-
	AndroidWorld	55.0	54.3	63.7	57.3	-	-	-	-
	OSWorld	30.6	30.3	41.0	32.6	-	-	-	-
	WindowsAA	24.2	24.9	42.9	30.9	-	-	-	-

多模态推理

- 旗舰模型在数学、逻辑等STEM任务（如MathVista、MathVision、ZeroBench）中表现领先，非思考模式（Instruct）和思考模式（Thinking）均优于同类模型。

- 中小型模型（如32B）在复杂推理任务上显著优于前代Qwen2.5-VL-72B，并超越Gemini-2.5-Flash和GPT-5-mini。